



Carnegie Mellon University

Master of
Software Engineering

LG 아키텍처 교육 프로그램 램

소프트웨어 아키텍처 프로젝트 소개: Timegrapher 프로젝트

이 강의의 내용 (및 다루지 않는 내용)

이 강의는 다음이 아닙니다:

- 상세한 요구사항 검토
- 코딩 튜토리얼
- 신호 처리 강의
- UI 디자인 검토 이 강의는

다음과 같습니다:

- 아키텍처 사고에 대한 논의
- 모호성에 대처하는 방법 안내
- 기술적 위험 관리 가이드
- 압박 속에서 올바른 아키텍처 결정을 내리는 가이드

왜 이 프로젝트인가?

- 실습을 통한 학습

- 이 프로젝트는 "아키텍처적 사고"를 연습할 기회를 제공합니다
- 이 프로젝트는 "코딩" 프로젝트가 아닙니다... 물론 코드를 분석하고 작성해야 할 필요는 있겠지만
- 이 프로젝트는 아키텍처상의 상충 관계를 분석하고 프로젝트 초기 단계에서 불확실성을 줄이는 데 중점을 둡니다

- 기술적 불확실성

- 초기에는 많은 기술적 불확실성이 존재할 것입니다
- 모든 영역이 아키텍처적 고려 사항에 있어 중요한 것은 아닙니다
- *중요한* 불확실성과 중요하지 않은 불확실성을 구별하는 방법을 배워야 합니다
- 또한 중요한 불확실성을 줄이는 방법을 배워야 합니다

이 프로젝트가 어려운 이유는 무엇일까요?

- 어려운 점은 특정 한 가지 요소 때문이 아닙니다
 - 비록 일부 요소가 복잡하긴 하지만
- 어려운 점은 다음 두 가지를 동시에 달성하는 것입니다:
 - 정확성
 - 실시간 성능
 - 다양한 기능을 갖춘 디스플레이
 - 노이즈에 대한 견고성
 - ...
- 이는 필연적으로 절충을 수반합니다 ...
 - (주어진 제약 조건 내에서) 최적의 절충점이 무엇인지 파악하는 것은 어렵습니다
 - 그리고 체계적인 접근 방식이 필요합니다

가장 큰 실수: 코딩의 함정

- 팀들이 흔히 사용하는 전략은 책임을 “하위 시스템” 단위로 구성하는 것입니다
 - 예: 데스크톱 앱, 라즈베리 파이, ...
- 그런 다음 팀은 책임을 세분화하고 코딩을 시작합니다
- 기본 기능이 완성되면 팀은 테스트를 시작한다 성능과 같은 QA 관련 문제

코딩 함정 II

- 왜 이것이 문제일까요?
 - 구현이 시작되면 QA 문제를 해결할 수 있는 선택지가 줄어듭니다
- 다시 말해, 솔루션을 재작업하거나 구현 과정에서 내린 결정을 그대로 받아들여야만 합니다
- 그 결정 중 일부는 (의도치 않게) 해결하려는 QA 문제에 필연적으로 영향을 미치게 된다

아키텍처 결정 요인

- 이 프로젝트는 기능만으로 정의되지 않습니다
- 사실, 품질 속성 결정 요인이 종종 더 중요하며(달성하기 더 어렵기도 합니다)
- 또한, 이와 같은 프로젝트에는 많은 제약 조건이 따릅니다
 - 일정과 자원은 분명 제약 조건이다
 - 하지만 기존 시스템 역시 여러분이 선택할 수 있는 옵션을 제한할 수 있습니다

타협은 피할 수 없습니다

- 품질 속성을 단독으로 달성하려는 경우는 거의 없습니다
- 일반적으로 여러 품질 속성을 동시에 달성하려고 합니다
 - 또한 품질 보증(QA) 문제에 영향을 미칠 여러 기능도 함께 달성하려 합니다
- 이는 거의 항상 상충 관계(한 가지 고려 사항을 충족시키면 다른 고려 사항이 저해되는 결정)를 초래합니다
- 최적의 절충점을 찾는 것이 핵심이다

불확실성 줄이기

- 이 프로젝트에는 많은 위험 요소가 있습니다:
 - 이 프로젝트는 상대적으로 복잡합니다
 - 새로 구성된 팀
 - 아마도 새로운 분야일 것입니다
 - 빠듯한 일정
- 많은 결정을 다시 내릴 시간이 없을 것입니다
- 이는 가정을 검증하고 불확실성을 조기에 줄여야 함을 의미합니다
- 이를 어떻게 할 수 있을까요?
 - 실험이 도움이 될 수 있습니다

실험

- 실험이란 무엇인가요?
 - 실험이란 아이디어, 가설 또는 방법을 테스트하여 그 효과를 확인하고, 가정을 검증하거나, 알려지지 않은 사실을 밝혀내는 과정입니다
- 이러한 종류의 프로젝트에서 이는 무엇을 의미할까요?
 - 먼저 프로젝트에서 무엇이 중요한지 이해하는 것에서 시작됩니다
 - 그런 다음 스스로에게 “기대를 충족시키기 위해 무엇이 필요한가?”라고 물어야 합니다
- 자연 시간이 중요한 문제라고 가정해 봅시다...

열린 질문

- 몇 가지 기본적인 질문이 떠오릅니다:
 - 충분한 지연 시간이란 무엇인가(그리고 어떤 성능이나 기능을 위한 것인가)?
 - 시스템은 어떤 부하 조건에서 원하는 지연 시간을 지원해야 하는가(부하 변화에 따라 지연 시간 요구 사항이 달라지는가)?
 - 현재 시스템은 (예상 부하 하에서) 어떤 지연 시간을 지원합니까?
 - 지연 시간을 개선하기 위한 옵션은 무엇인가?
 - 이러한 옵션들은 얼마나 효과적인가(그리고 어떤 장단점이 있는가)?
 - 다양한 옵션을 구현하는 데 드는 노력은 어느 정도인가요?

실험의 우선순위 결정

- 실험에는 분명히 시간과 노력이 소요됩니다
 - 당신은
 - 모든 것을 다 할 시간은 없을 것입니다
- 실험을 선정하고 우선순위를 정하는 것은 일종의 기술이라 할 수 있습니다
 - 다른 자원 배분과 마찬가지로 이 문제도 신중하게 고려해야 합니다
 - 비용과 이익(ROI)의 관점에서 생각해 보십시오
 - 다음과 같이 자문해 보세요: 이 실험에 얼마나 많은 노력이 필요할까요? 결과에서 기대되는 가치는 얼마인가요(다른 실험과 비교했을 때)?

팀 조직

- 팀 구성으로 돌아가기...
- 팀을 어떻게 구성해야 할까요?
 - 정의한 업무를 중심으로 팀을 구성해야 합니다
 - 팀 조직 구조에서 업무가 파생되어서는 안 됩니다
- 즉, 팀 구조를 정하기 위해서는 계획에 대한 어느 정도의 구상이 필요합니다
 - 계획은 단계별로 구성될 수 있습니다(한 단계에서 다른 단계로 넘어갈 때 업무의 성격이 변화하는 경우).
- 이는 팀 구조 또한 변화할 수 있음을 의미합니다...

기타 위험 요소...

- 상당한 규모의 역량을 구축해야 하는 상황입니다
 - 이 또한 지연을 초래할 수 있는 수많은 다른 위험 요소들이 수반됩니다
- 주어진 기간 내에 실제로 얼마나 많은 작업을 완료할 수 있을지는 (우리에게도) 불분명합니다
- 즉, 모든 작업을 완료하지 못할 위험이 있다는 뜻입니다
 - 하지만 이러한 위험을 관리하기 위한 알려진 전략들이 있습니다
 - 점진적 제공이 그러한 전략 중 하나입니다
- 이는 반복적인 방식으로 작업을 진행할 계획임을 의미합니다
 - 매 반복마다 시스템의 작동 가능한 버전을 전달하는 방식입니다

스튜디오 프로젝트에 대한 기대

- 열심히 일해야 할 것이지만, 그만큼 보람 있는 경험이 될 것입니다.
 - 대부분의 주말에는 수업 및 프로젝트 과제를 위해 작업해야 합니다.
 - 수업 자료를 최대한 활용하도록 노력해 주시기 바랍니다.
 - 멘토들이 강의 내용을 실제에 적용하는 데 도움을 줄 것입니다 –
꼭 활용하세요!
 - 마지막 주에 급하게 해결책을 짜내지 마세요. 새로운 디자인 기법과 개념을 적용해 보세요!
 - 끊임없이 되돌아보세요: 무엇이 잘되었는지, 무엇을 더 잘할 수 있었는지, 다음에는 무엇을 할 지.

성과물 I

- 이 프로젝트에는 3개의 마일스톤이 있습니다
- 각 마일스톤별 산출물은 다음과 같습니다:
 - 마일스톤 1:
 - 개발 계획
 - 아키텍처 결정 요인
 - 아키텍처 접근 방식 초안
 - 확인된 위험 요소 목록
 - 계획된 실험/초기 실험 결과

산출물 II

- 마일스톤 II

- 업데이트된 개발 계획
- 업데이트된 설계
- 잔여 기술적 리스크
- 실험 계획/결과

- 마일스톤 III

- 시스템 데모
- 최종 발표

프로젝트 평가

- 프로젝트의 각 단계별로 평가가 이루어집니다
- 배점은 다음과 같습니다:
 - 마일스톤 I: 25%
 - 마일스톤 II: 25%
 - 마일스톤 III: 50%
- 마일스톤 I 및 II의 경우 다음 사항을 평가합니다:
 - 팀이 아키텍처 관행을 얼마나 잘 구현했는지
 - 드라이버가 실행 가능한지
 - 드라이버가 시스템의 목표를 반영하고 있는가
 - 팀이 드라이버에 영향을 미칠 설계 결정을 식별했는지
 - 팀이 위험을 파악하기 위해 핵심 영역을 충분히 조사했는지
 - 팀이 설계의 핵심적인 측면을 이해하고 있는가
 - 개발 계획이 전반적인 목표를 얼마나 잘 뒷받침하고 있는가
 - 구현된 시스템이 의도된 아키텍처를 준수하는가

프로젝트 데모

- 이번 데모에서는 다음 기준을 살펴보겠습니다:
 - 필수 기능의 구현 여부
 - 시스템의 견고성/정확성
 - 분석 속도
 - 표시 속도
 - 측정 정확도
 - 오류 감지

이용 가능한 리소스

- 팀 지원을 위한 자료 제공
- 각 팀은 다음을 이용할 수 있습니다:
 - 멘토
 - Dan Plakosh, Jason Popowski, Stephan Beck (제품 소유자)
- 댄, 제이슨, 스티브는 스타터 코드를 개발했으며 기술, 위험 요소 및 요구 사항을 잘 알고 있습니다
 - 이와 관련된 구체적인 사항에 대한 문의는 댄, 제이슨, 스티브에게 하시기 바랍니다
- 멘토들은 프로젝트 실행을 지원할 수 있습니다
 - 그들은 기꺼이 계획과 전략을 검토하고, 아키텍처 관행의 실행을 안내하며, 아키텍처 관련 조언과 지원을 제공할 것입니다

제안 사항

- 사전 작업에 대해 신중하게 고려하십시오
 - 실험을 통해 기술적 위험을 파악하고 설계 가정을 검증해야 합니다
- 프로젝트의 목표가 명확히 이해되었는지 확인하십시오
 - “성공”은 어떻게 평가될 것인가?
 - 이러한 위험 요소나 실험이 성공 기준과 관련이 있는가?
- 작업 분할에 대해 신중하게 고려하십시오
 - 작업은 어떻게 배정될 것인가?
 - 작업에 대한 “완료 기준”은 무엇인가요?
 - 작업들은 어떻게 상호 의존적입니까?
 - 정보가 필요할 때, 필요한 방식으로 교환되도록 어떻게 보장할 것인가?

